

[자연어처리와 컴퓨터비전 심층학습]

7장

트랜스포머

- 기존의 순환 신경망과 같은 순차적 방식이 아닌 병렬로 입력 시퀀스 처리
- 순차 처리, 반복 연결에 의존하지 않고 셀프 어텐션을 기반으로 함
→ 긴 시퀀스의 경우 훨씬 더 빠르고 효율적인 처리 가능
- 오토 인코딩과 자기 회귀 방식(디코더) 또는 두 개의 조합으로 학습
- 용어 정리
 - 인코더: 예측되는 토큰 양옆에 있는 토큰들을 참조하는 양방향 구조
 - 디코더: 예측되는 토큰의 왼쪽에 있는 토큰들만 참조하는 단방향 구조

Transformer

- 구조

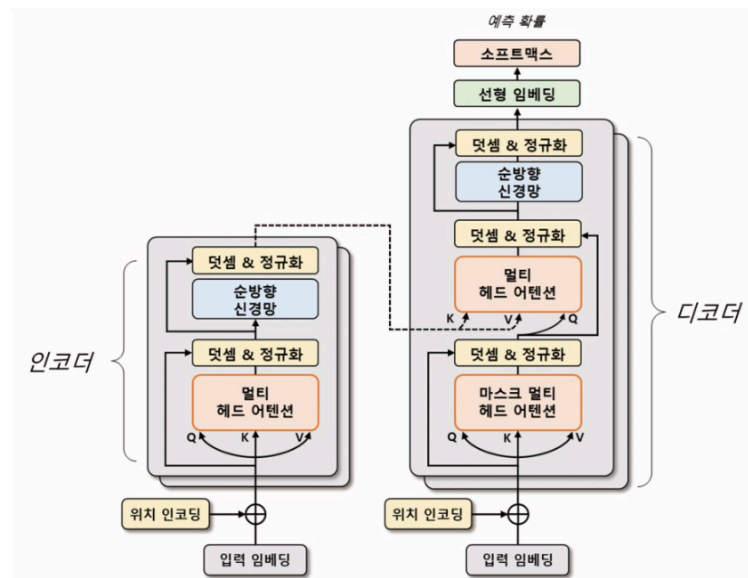


그림 7.2 트랜스포머 모델 구조(Q: 쿼리, K: 키, V: 값)

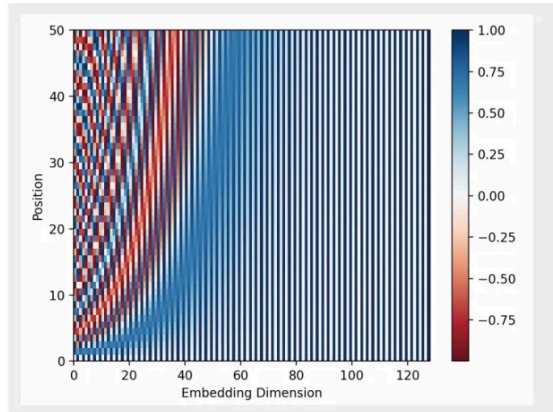
- 인코더와 디코더는 각각 N개의 트랜스포머 블록으로 구성됨
- 이 블록은 멀티 헤드 어텐션과 순방향 신경망으로 이루어짐

- 멀티 헤드 어텐션
 - 쿼리, 키, 값 벡터를 정의해 입력 시퀀스들의 관계를 셀프 어텐션으로 표현
 - 쿼리와 각 키의 유사도를 계산하고, 해당 유사도를 가중치로 사용해 값 벡터 합산
- 순환 신경망
 - 산출된 임베딩 벡터를 고도화하기 위해 사용
 - 여러 개의 선형 계층으로 구성
 - 벡터에 가중치를 곱하고, 편향을 더하며, 활성화 함수를 적용
 - 학습된 가중치들은 입력 시퀀스와 각 단어의 의미를 잘 파악할 수 있는 방식으로 갱신
 - 입력 시퀀스 데이터를 소스와 타겟 데이터로 나눠 처리
 - 인코더 - 소스 시퀀스 데이터를 위치 인코딩된 입력 임베딩으로 표현해 출력 벡터 생성
 - 디코더 - 마스크 멀티 헤드 어텐션을 사용해 타겟 시퀀스 데이터 순차적으로 생성

입력 임베딩과 위치 인코딩

- 트랜스포머 모델은 입력 시퀀스를 병렬 구조로 처리해 단어의 순서 정보를 제공하지 않음
- 따라서 위치 정보를 임베딩 벡터에 추가해 단어의 순서 정보를 모델에 반영해야 함
 - 위치 인코딩 방식 사용
- 위치 인코딩
 - 입력 시퀀스의 순서 정보를 모델에 전달하는 방법
 - 각 단어의 위치 정보를 나타내는 벡터를 더하여 임베딩 벡터에 위치 정보 반영
 - 각 토큰의 위치를 각도로 표현해 $\sin$ 함수와 $\cos$ 함수로 위치 인코딩 벡터 계산
 - 토큰의 위치마다 동일한 임베딩 벡터를 사용하지 않아 각 토큰의 위치 정보를 모델이 학습 가능

출력 결과

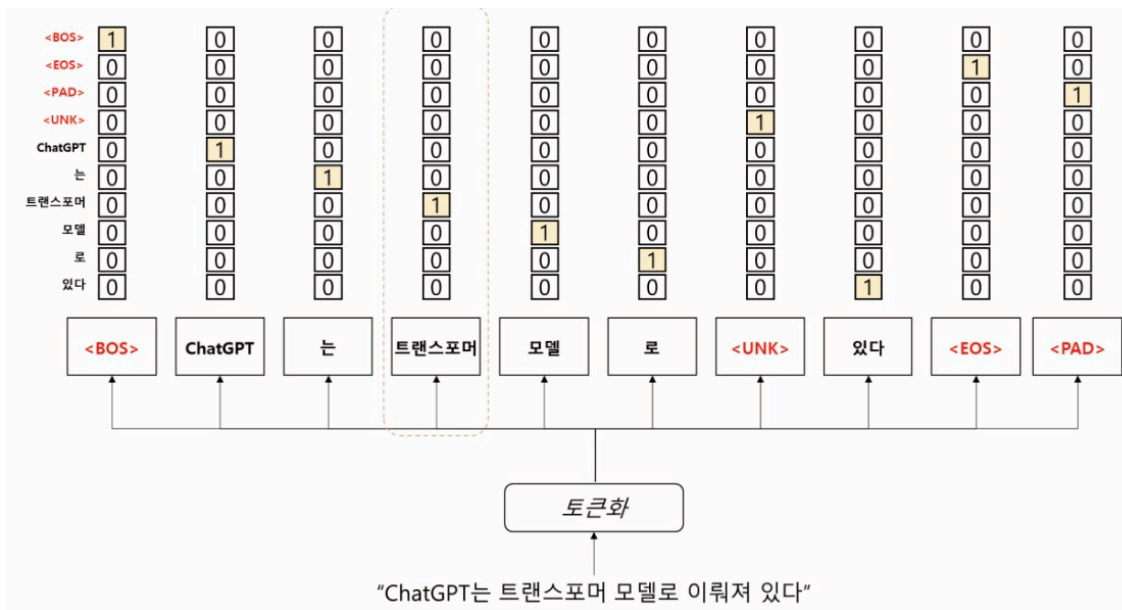


$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$

$$PE(pos, 2i + 1) = \cos(pos/10000^{2i/d_{model}})$$

특수 토큰

- 입력 시퀀스의 시작과 끝을 나타내거나 마스킹 영역으로 사용되어 문장을 표현



BOS(Beginning of Sentence), EOS(End of Sentence), UNK(Unknown), PAD(Padding)이 특수 토큰

- BOS: 문장의 시작
- EOS: 문장의 종료
- UNK: 어휘 사전에 없는 모르는 단어
- PAD: 모든 문장을 일정한 길이로 맞추기 위해 사용
- 생성된 문장 토큰 배열은 원-핫 인코딩으로 표현됨

트랜스포머 인코더

- 입력 시퀀스를 받아 여러 개의 계층으로 구성된 인코더 계층을 거쳐 연산 수행
- 헤드 어텐션과 순방향 신경망으로 구성
- 입력 데이터에 대한 정보를 추출하고 다음 계층으로 전달
- 연산 과정
 1. 위치 인코딩이 적용된 소스 데이터의 입력 임베딩 입력 받음
 2. 선형 변환을 통해 3개의 임베딩 벡터 생성
 3. 생성된 벡터를 각각 쿼리(Q), 키(K), 값(V) 벡터로 정의

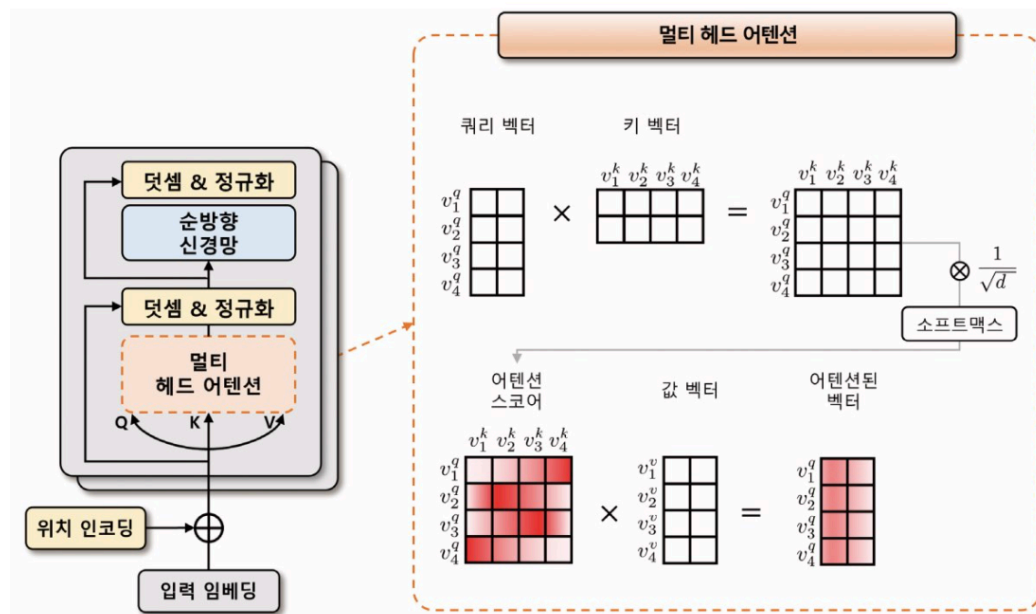


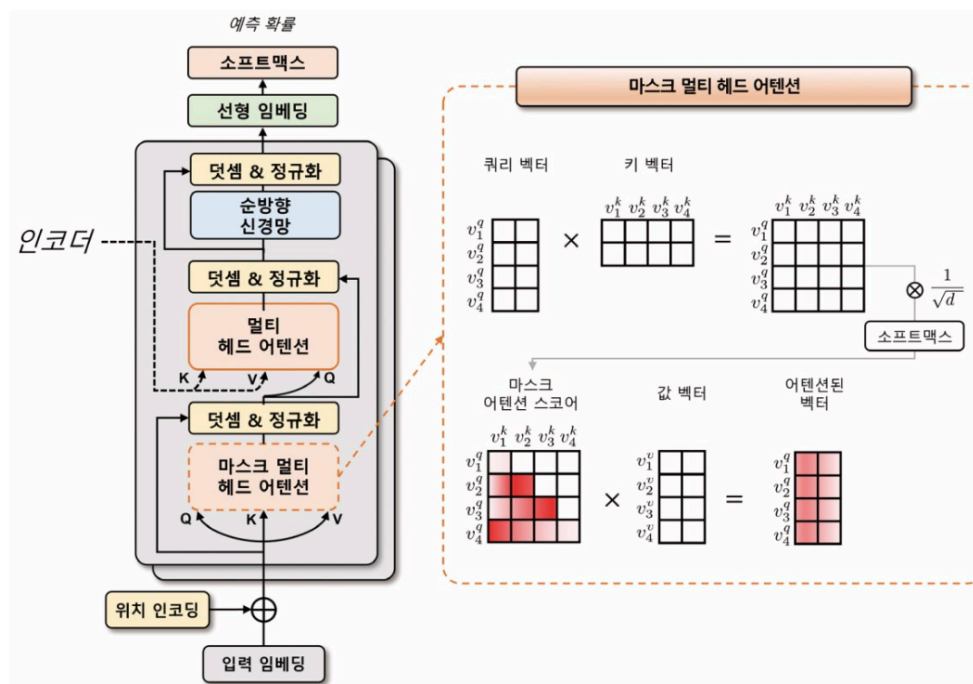
그림 7.5 트랜스포머 인코더 블록 연산 과정

- 쿼리 벡터: 현재 시점에서 참조하고자 하는 정보의 위치를 나타내는 벡터
- 키 벡터: 쿼리 벡터를 제외한 입력 시퀀스에서 탐색되는 벡터
- 값 벡터: 쿼리 벡터와 키 벡터로 생성된 어텐션 스코어를 얼마나 반영할지 설정하는 가중치 역할을 수행하는 벡터

$$score(v^q, v^k) = \text{softmax}\left(\frac{(v^q)^T \cdot v^k}{\sqrt{d}}\right)$$

트랜스포머 디코더

- 위치 인코딩이 적용된 타겟 데이터의 입력 임베딩을 입력받음
- 인코더의 멀티 헤드 어텐션 모듈이 인과성을 반영한 마스크 멀티 헤드 어텐션 모듈로 대체
- 마스크 멀티 헤드 어텐션 모듈
 - 첫 번째 쿼리 벡터가 첫 번째 키 벡터만을 바라볼 수 있게 마스크 씌움
 - 두 번째 쿼리 벡터는 첫 번째와 두 번째 키 벡터를 바라보게 마스크 씌움
→ 셀프 어텐션에서 현재 위치 이전의 단어들만 참조할 수 있게 되며 인과성 보장
 - 마스크 영역에 수치적으로 굉장히 작은 값은 $-\text{inf}$ 마스크를 더해 해당 영역의 어텐션 스코어값이 0에 가까워지도록 함
→ 소프트맥스 계산 시 마스크 영역 이전에 있는 입력 토큰들에 대한 정보를 참조하지 않아도 됨
- 연산 과정
 1. 타겟 데이터가 쿼리 벡터로, 인코더의 소스 데이터가 키와 값 벡터로 사용
 2. 쿼리, 키, 값 벡터를 이용해 어텐션 스코어 맵을 계산하고 소프트맥스 함수를 적용해 어텐션 가중치 구함
 3. 어텐션 가중치와 값 벡터를 가중합하여 멀티 헤드 어텐션의 출력 벡터를 얻음



GPT

- 트랜스포머의 디코더를 여러 층 쌓아 만든 언어 모델
- 인코더는 입력 문장의 특징을 추출하는데 초점을 둔다면, 디코더는 입/출력 문장 간 관계를 이해하고 출력 문장을 생성하는 데 초점
→ 디코더를 쌓아 모델을 구성하면 자연어 생성과 같은 언어 모델링 작업에서 더 높은 성능 발휘
- 대규모 텍스트 데이터셋으로 사전 학습해 초기화, 특정 자연어 처리 작업에 맞게 미세 조정해 사용

GPT-1

- 정리
 - 단방향 언어 모델: 텍스트를 생성할 때 현재 위치에서 이전 단어들에 대한 정보만 참고해 다음 단어 예측
 - 이전 단어들을 차례대로 입력하며 다음 단어를 예측하기 때문에 학습 속도가 빠르고 모델의 크기가 작아짐

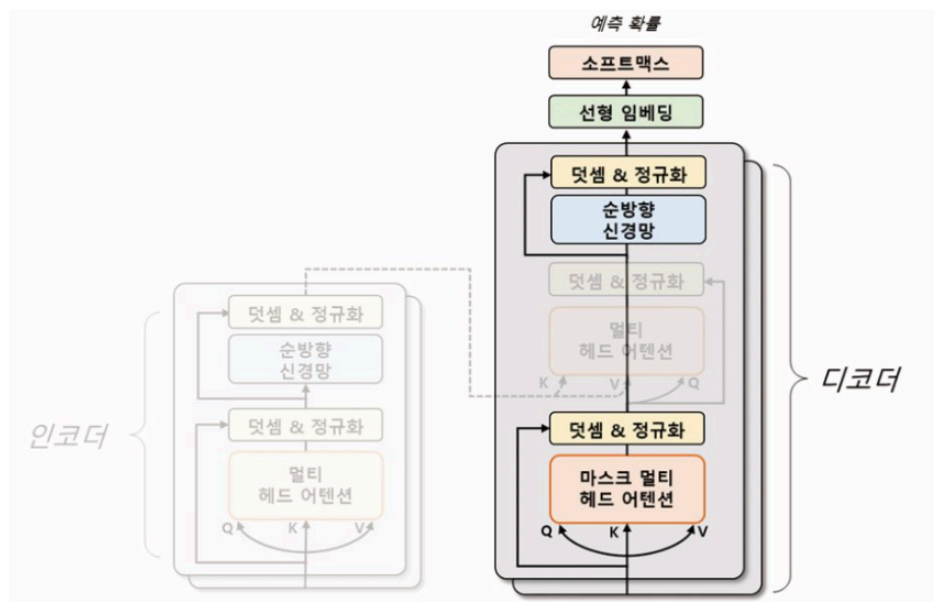


그림 7.7 트랜스포머와 GPT-1 모델 비교

- 사전 학습 구조
 - 입력 문장의 일부만 보고 다음 토큰을 예측하도록 하는 언어 모델을 통해 사전 학습
 - 입력 문장을 일정한 길이의 블록으로 나누어 처리
 - 각 블록에서 블록 내 첫 번째 토큰부터 순서대로 다음 토큰 예측

→ 이러한 방식은 모델이 장기적인 문맥을 이해할 수 있게 함

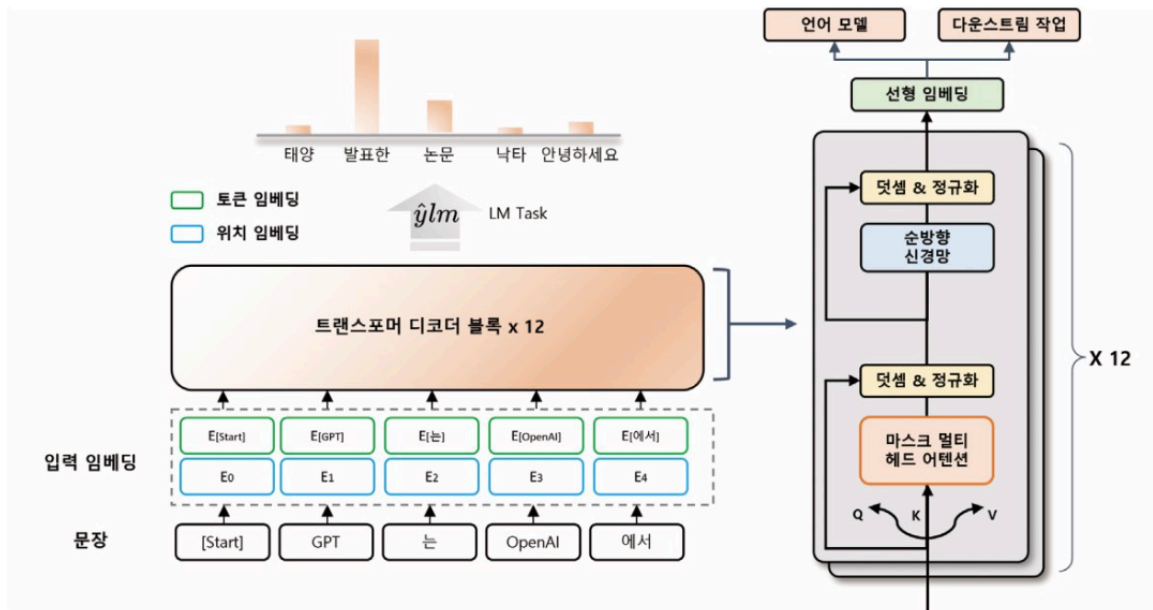


그림 7.8 GPT-1의 사전 학습 작업 구조

- 입력 임베딩
 - 토큰 임베딩: 각 토큰을 고정된 차원의 벡터로 매핑
 - 위치 임베딩: 입력 문장에서 각 토큰의 위치 정보를 나타내는 벡터를 매핑
- 보조 학습
 - 정의: 모델이 주어진 학습 작업 외에도 다른 작은 부가적인 학습 작업을 동시에 수행하는 것
 - 보조 작업에서 학습한 정보를 주요 작업에 전달해 성능 향상시킬 수 있음
 - GPT-1은 다운스트림 작업에서 입출력 구조를 바꾸지 않고 보조 학습해 사용함

GPT-2

- GPT-1과의 차이

	GPT-1	GPT-2
최대 입력 문장 길이	512	1024
디코더 층 개수	12개	48개
매개변수 개수	1200만개	15억개

- 모델 구조

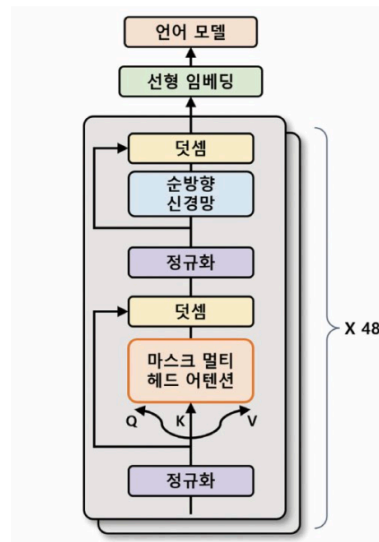


그림 7.10 GPT-2의 모델 구조

GPT-3

- GPT-2와의 차이
 - GPT-2보다 116배 많은 매개변수를 가짐
- 정리
 - 규모가 매우 큰 모델
 - 96개의 헤드를 가진 멀티 헤드 어텐션 사용
 - 96개의 트랜스포머 디코드 층 사용
 - GPT-2와 같이 프롬프트 형식으로 작업 수행
- 단점
 - 큰 모델이어서 느리고 비용이 많이 듦
 - 사전 학습된 데이터에 기반하여 작동해 데이터가 편향되어 있거나 정확하지 않은 경우 오류가 발생
 - 인간적인 이해력과 상식적인 추론 능력 부족

GPT 3.5

- 정리
 - GPT-3의 모델 구조를 그대로 따르면서도 새로운 학습 방법인 RLHF 도입
 - 매개변수 수를 줄이고 모델의 자연스러움을 높임

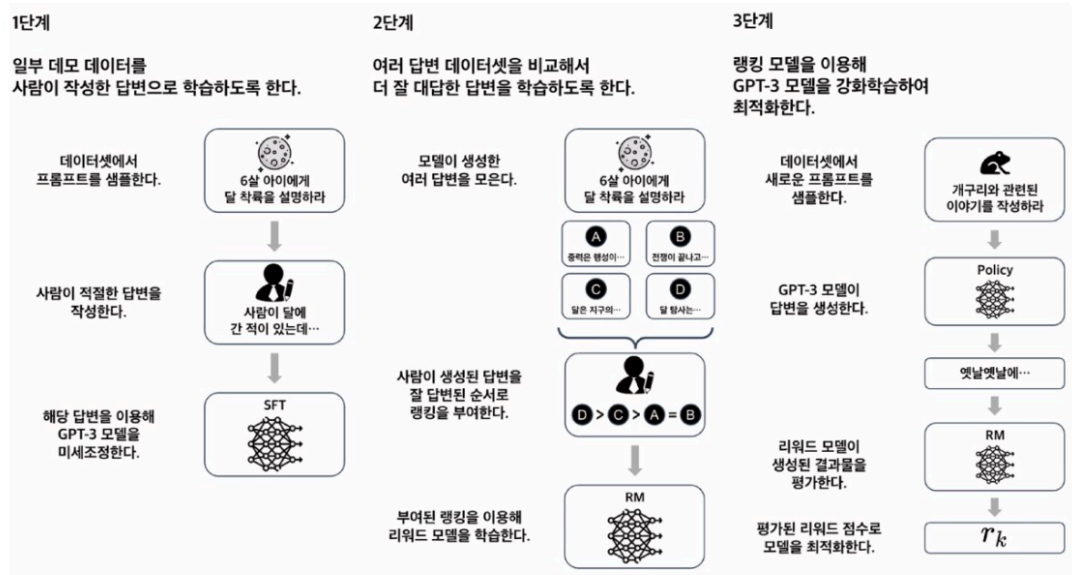


그림 7.12 RLHF GPT-3 학습 방법

○ 지도 미세 조정

- 모델이 생성한 문장이 자연스러우면 보상, 그렇지 않으면 보상받지 못함
- 이러한 보상을 바탕으로 강화 학습을 통해 모델 학습

○ 보상

- 학습 데이터양을 고려했을 때 전체 프롬프트에 사람이 답변을 다는 것은 현실적으로 어려움
- 따라서 모델이 얼마나 잘 답변했는지 평가하는 보상 모델 학습

○ 한계

- 여전히 환각 현상 존재

*환각 현상: 인간과 같은 추론과 판단 능력을 갖추지 못하고 부적절하거나 오류가 많은 답변을 제시하는 현상

GPT-4

• 정리

- 이전의 GPT 모델과 달리 이미지 데이터도 인식 가능한 멀티모달 모델
- 가장 많은 토큰을 처리할 수 있으며, 가장 높은 성능을 보임